



Agentes e Sistemas Multiagente

Gestão de Veículos de Emergência Médica

Sistemas Inteligentes
Mestrado em Informática Médica

Braga, janeiro 2024

Docentes:

Paulo Jorge Freitas Oliveira Novais;

Pedro José Costa Oliveira.

Contribuições:

Mariana Ribeiro, pg54061 = 0;

Mariana Almeida, pg54062 = 0;

Madalena Passos, pg54023 = 0;

Mónica Martins, a95918 = 0.

1. Índice

1. Índice.....	2
2. Introdução	3
3. Agentes e Behaviours Associados.....	4
3.1 Agente UGVE	4
3.1.1 ReceiveRequestBehav	5
3.2 Agente Doente.....	7
3.2.1 SendRequest_behav	7
3.2.2 DoenteRecebeBehav	7
3.3 Agente UnidadeSaude	7
3.3.1 RegisterUS_Behav	8
3.3.2 UsGreve.....	8
3.3.3 us_MudaEstado	9
3.3.4 usRecebeDoente	9
3.4 Agente Veículo	9
3.4.1 RegisterVeiculo_Behav	10
3.4.2 Transport_Behav	10
4. Arquitetura do sistema Multiagente	11
4.1 Diagrama de Classes	12
4.2 Diagrama de Colaboração	13
4.3 Diagrama de Sequência.....	15
4.4 Diagrama de Atividades	17
5. Implementação do Sistema MultiAgente	18
6. Conclusão e Melhorias Futuras	19
7. Referências Bibliográficas	19

2. Introdução

No âmbito da temática de Agentes e Sistemas Multiagente [1], foi proposto o desenvolvimento de um projeto focado na gestão eficiente de veículos hospitalares em situações de emergência. Desta forma, foi necessário desenvolver agentes inteligentes capazes de perceber e interpretar informações de um ambiente virtual, coordenando decisões de forma colaborativa.

O cenário simulado envolve uma Unidade de Gestão de Veículos de Emergência (UGVE) responsável por gerir diferentes veículos ao receber pedidos de transporte de doentes. Ao receber um pedido, a UGVE irá selecionar o veículo mais adequado com base no estado do doente e na distância até à geolocalização do mesmo. O veículo, ao receber a solicitação, deverá deslocar-se até ao doente, atualizando o estado atual para ocupado, e posteriormente, transportá-lo para uma Unidade de Saúde (US) designada pela UGVE (tendo em consideração requisitos como a distância, especialidades existentes e vagas disponíveis e existência ou não de heliporto).

O Sistema MultiAgente (SMA) a ser desenvolvido terá como principais objetivos:

- Definição dos Agentes, Classes e Behaviours e Performatives do sistema Multiagente;
- Gestão Eficiente de Veículos Hospitalares em situações de emergência;
- Simulação Realista do processo de transporte de doentes, considerando variáveis como o estado do paciente, tipos de veículos de emergência e capacidade das unidades de saúde hospitalares;
- Elaboração de planos de contingência, implementando medidas a adotar em situações específicas, como a indisponibilidade de heliportos em unidades hospitalares.

Além destas funcionalidades base, o sistema tem ainda em consideração a possibilidade de greves nas USs adotando, portanto, medidas para gestão destas situações.

O desenvolvimento do projeto compreende a descrição de arquitetura distribuída baseada em agentes utilizando Agente UML. Esta inclui a definição de características e tipos de agentes, a arquitetura do sistema Multiagente e outras funcionalidades relevantes. A implementação dos agentes será realizada utilizando a biblioteca SPADE [2], utilizando a linguagem de programação Python.

3. Agentes e Behaviours Associados

Numa etapa inicial do projeto, de forma a aprofundar a compreensão dos objetivos propostos, foi desenvolvido um esquema elucidativo que ilustra, de forma geral, o papel dos agentes e as funções que estes irão desempenhar no sistema.

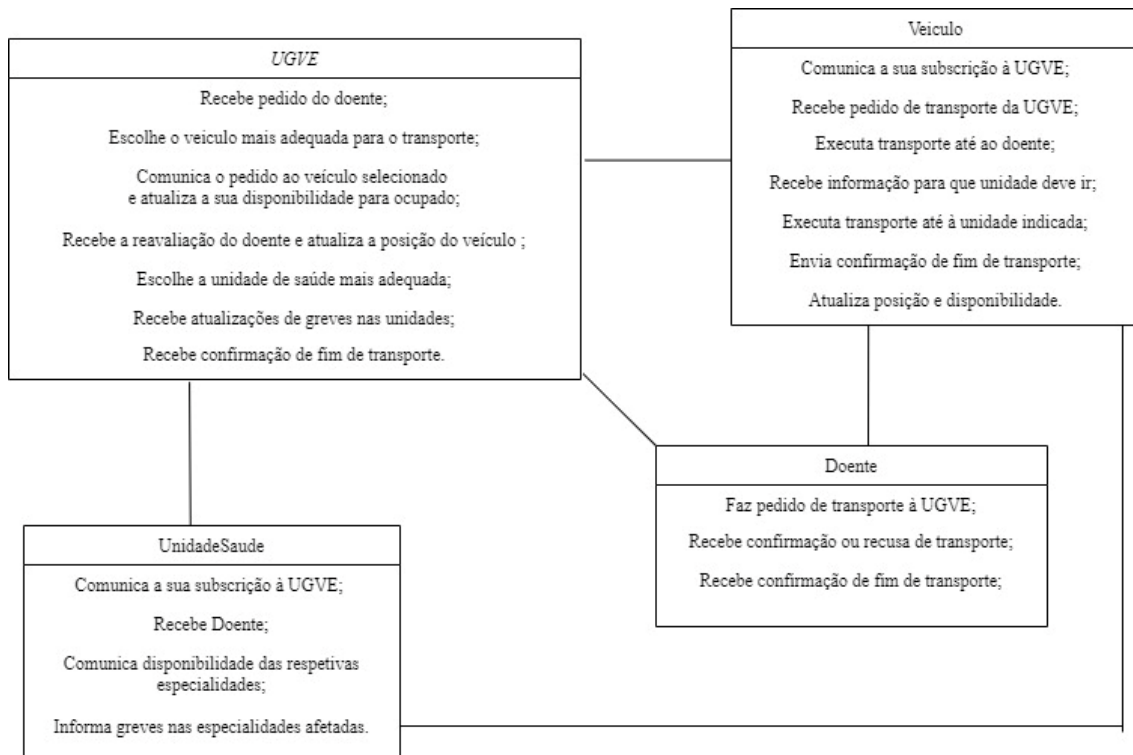


Figura 1-Esquema inicial referente à definição e representação das funções dos agentes desenvolvidos.

De acordo com a Figura 1, foram implementados quatro agentes **UGVE**, **UnidadeSaude**, **Veiculo** e **Doente**, cujos *Behaviours* associados irão ser explicados mais em detalhe de seguida.

3.1 Agente UGVE

A UGVE desempenha um papel central na gestão de todo o processo, atuando de forma a permitir a coordenação entre os diferentes agentes no sistema. Assim, este agente recebe informações do Agente Veiculo e do Agente UnidadeSaude por meio de objetos, guardando-os em listas, *veiculos_subscribed* e *unidades_subscribed*, respetivamente.

Posteriormente, é feito um pedido de transporte pelo Agente Doente, pelo que a UGVE irá processar o pedido, atribuindo ao doente o veículo mais adequado e a unidade de saúde mais

apropriada. Desta forma, a UGVE implementa apenas um *CyclicBehaviour* que garante tanto a receção como o envio contínuo de informações e pedidos dentro do sistema.

3.1.1 ReceiveRequestBehav

No contexto do *behaviour ReceiveRequestBehav*, o agente inicia o processo ao receber um objeto proveniente do agente UnidadeSaude com informações, utilizando a *performative subscribe_unidade*. De seguida, após a decodificação da mensagem, a UGVE procede ao registo da unidade de saúde associada, armazenando o objeto correspondente na lista *unidades_subscribed*. Recorrendo ao mesmo processo, agora associado à *performative subscribe_Veiculo*, efetua-se o registo dos veículos de emergência, guardando o objeto relativo a cada um na lista *veiculos_subscribed*.

Ao receber a solicitação de transporte efetuada pelo doente, a UGVE procede ao processamento deste, sendo o mesmo iniciado pela *performative request_veiculo*.

Verificando-se este cenário, em primeiro lugar, a mensagem relativa ao pedido é decodificada, sendo selecionado o tipo de transporte de acordo com o estado do doente, correspondendo à ambulância da cruz vermelha o estado leve, ambulância dos bombeiros para moderado, ambulância INEM para grave ou muito grave, e helicóptero para situações urgentes.

Após ser determinado qual o tipo de veículo mais adequado para as circunstâncias, é efetuada a procura pelo veículo correspondente disponível e mais próximo da localização do doente. Se for encontrado um veículo disponível nas proximidades, este é selecionado e é enviada uma mensagem ao doente, com a confirmação de transporte (*performative confirm_transport*) e o veículo responsável pelo mesmo. Adicionalmente, são atualizados os atributos disponibilidade para *false* e posição do veículo para a localização do doente.

No caso de não ser encontrado nenhum veículo próximo disponível, é enviada ao doente uma mensagem informando a situação, através da *performative refuse*.

Por último, o agente recebe uma mensagem com a *performative inform_reavaliacao* proveniente do veículo que executou o transporte até ao geolocalização do doente. Esta mensagem inclui um objeto com informações acerca da localização e da especialidade para a qual o doente deverá ser encaminhado.

Por sua vez, a UGVE processa a reavaliação recebida, verificando quais unidades de saúde têm a especialidade indicada e se existem vagas disponíveis para responder ao pedido. No caso de o veículo responsável ser um helicóptero, o transporte ficará restrito às unidades de saúde equipadas com heliporto. De notar que ao utilizar um helicóptero como meio de transporte, o doente deverá ser encaminhado para a especialidade de urgência, especialidade essa que se encontra em todas

as unidades equipadas com heliporto. Desta forma, a UGVE irá selecionar a unidade de saúde com heliporto mais próxima, que já contém a referida especialidade. No entanto, uma vez que existem 2 unidades de saúde com heliportos e 2 helicópteros em serviço, em caso de indisponibilidade de 1 heliporto, o helicóptero será reencaminhado para o outro (que estará disponível). De seguida, a UGVE enviará uma mensagem para o veículo com a *performative inform_unidade*, contendo como objeto a unidade de saúde mais próxima com heliporto disponível. Assim, o veículo tomará conhecimento para que unidade de saúde deve dirigir-se, prosseguindo com o transporte. Adicionalmente, a UGVE atualiza o número de vagas disponíveis na especialidade da unidade de saúde selecionada, assegurando a precisão das informações sobre a capacidade de atendimento. Posteriormente, também atualiza na lista de *veiculos_subscribed* a posição do veículo para a localização da US, para este poder ser utilizado novamente de forma correta.

Em relação aos restantes tipos de veículos, estes podem deslocar-se para qualquer unidade de saúde registada independentemente da existência ou não de heliporto, sendo a escolha da mesma realizada de forma idêntica, tendo apenas como critério a distância, a presença e disponibilidade da especialidade requerida.

No final do transporte do utente para a US selecionada, a disponibilidade do veículo é atualizada para *true*, pelo que passa a estar preparado para efetuar um novo transporte, recorrendo para tal à *performative confirm_end*.

Em situações de greve de algumas especialidades nas unidades de saúde, a UGVE recebe a notificação de greve do Agente UnidadeSaude por meio da *performative muda_estado*. Após receber essa informação, realiza alterações na lista correspondente, atualizando as vagas disponíveis das especialidades afetadas para 0. De forma semelhante, a UGVE também recebe uma mensagem do Agente UnidadeSaude indicando que foram feitas atualizações no número de vagas disponíveis em cada especialidade face à entrada ou saída de doentes, sendo esta útil para que a UGVE proceda à atualização da informação guardada.

Em resposta à possível limitação no número de vagas nas unidades, foram implementadas algumas medidas de contingência de forma a contornar a situação. Desta forma, caso não existam vagas disponíveis em nenhuma das unidades que asseguram a especialidade indicada na reavaliação do doente, este deverá ser reencaminhado para uma lista de espera das unidades de saúde *us1* ou *us2*.

3.2 Agente Doente

O agente Doente foi desenvolvido com o objetivo de representar, como o nome indica, o paciente. Para a implementação deste agente, foi necessário desenvolver dois *behaviours*: *SendRequest_behav* e *DoenteRecebeBehav*.

3.2.1 SendRequest_behav

O primeiro *behaviour* trata-se de um *OneShot Behaviour* e nele efetua-se a criação aleatória de uma instância da classe *Position*, com recurso à biblioteca *random*. A mesma biblioteca foi utilizada para escolher aleatoriamente o estado do paciente, dentro de uma lista de estados possíveis (*estadosposs=["leve", "moderado", "grave", "muito grave", "urgente"]*).

Selecionados os parâmetros, é criada uma instância da classe *MakeRequestDoente*, composta pelo *jid* do doente bem como localização e estado do mesmo. Por fim, este *behaviour* é responsável pelo envio de uma mensagem à UGVE, com a *performative request_veiculo*, cujo *body* trata-se da instância *MakeRequestDoente* criada (e codificada com a biblioteca *jsonpickle*).

3.2.2 DoenteRecebeBehav

O *behaviour DoenteRecebeBehav* é um *behaviour* do tipo *Cyclic*, criado para suportar as mensagens com diferentes *performatives* que lhe podem ser enviadas. Para isso, espera-se a chegada de uma mensagem a cada 10 segundos (*msg = await self.receive(timeout=10)*). Caso o agente receba uma mensagem, verifica-se a *performative* da mesma (com recurso à função *get_metadata("performative")*). Se a *performative* for do tipo *confirm* é efetuado um print que informa a realização do transporte do doente com sucesso. De forma semelhante, se a *performative* recebida for do tipo *refuse*, é mostrada a informação de que não existem veículos disponíveis. Se for recebida qualquer outra *performative*, o agente comunicará a falta de compreensão da mesma.

3.3 Agente UnidadeSaude

Para representar as Unidades de Saúde, foi desenvolvido o agente UnidadeSaude. Este agente é composto por quatro *behaviours*: *RegisterUS_Behav*, *usRecebeDoente*, *usMudaEstado* e *usGreve*.

3.3.1 RegisterUS_Behav

Este *behaviour* é um *OneShot Behaviour* e foi desenvolvido com o objetivo de permitir que a Unidade de Saúde enviasse as suas informações à UGVE (por exemplo, a localização).

Neste *behaviour* deve criar-se uma instância da classe *InformEstadoUS*, que é composta pelo *jid* do agente *UnidadeSaude*, localização, especialidades, (existência ou não) heliporto e a disponibilidade do mesmo.

Para definir a existência ou não de heliporto, foi definido que as Unidades de Saúde com os nomes *us1* e *us2* se tratariam de hospitais com melhores capacidades e condições e que, por isso, teriam heliporto. Pelos mesmos motivos, foi definido que estes hospitais teriam todas as especialidades, bem como uma lista de espera para lidar com situações de greve ou indisponibilidade da especialidade.

Para definir as especialidades das restantes Unidades de Saúde, foi criada uma lista de especialidades possíveis que as Unidades de Saúde poderiam ter e inicializou-se um dicionário vazio. Este dicionário (especialidades) é composto pelas especialidades existentes (*keys*) e as vagas existentes na mesma (*values*). Para preencher o dicionário, selecionou-se um número aleatório de especialidades que a Unidade de Saúde teria e efetuou-se um ciclo *for* com tantos ciclos quanto o número selecionado. A cada ciclo, seleciona-se aleatoriamente uma especialidade (dentro das possíveis) e o número de vagas para a mesma.

Definidos todos os parâmetros, é criada uma instância do objeto *InformEstadoUS*, com as informações relativas ao agente e, consequentemente, à Unidade de Saúde em questão. Esta instância é guardada como um atributo do agente, na variável *current_state*.

Por fim, é enviada uma mensagem ao Agente UGVE, com a *performative subscribe_UnidadeSaude*, com o objeto criado e codificado com a biblioteca *jsonpickle*.

3.3.2 UsGreve

O *behaviour usGreve* pretende simular as situações de greve cada vez mais recorrentes nos dias de hoje nas Unidades de Saúde. Assim, este trata-se de um *TimeoutBehaviour* que após um determinado período de tempo executa as ações pretendidas. Neste caso, o *behaviour* atualiza a disponibilidade de algumas especialidades escolhidas aleatoriamente para 0, mimetizando as situações nas quais os serviços de algumas especialidades se encontram indisponíveis nas USs. A escolha das especialidades em situação de greve é feita aleatoriamente a partir da lista de especialidades existentes nessa Unidade de Saúde.

De forma a manter a informação coerente em todo o sistema Multiagente, é enviada uma mensagem para a UGVE com as informações de disponibilidade atualizadas.

3.3.3 *us_MudaEstado*

O *behaviour us_MudaEstado* trata-se de um *behaviour* periódico, isto é, *PeriodicBehaviour*. À semelhança do *behaviour* anterior, este *behaviour* permite atualizar as disponibilidades dos serviços de especialidade nas Unidades de Saúde. No entanto, ao contrário do *behaviour usGreve*, o *us_MudaEstado* gera automaticamente valores entre 0 e 10 de vagas disponíveis para todas as especialidades existentes na Unidade de Saúde.

Este *behaviour* pretende simular a natureza dinâmica das Unidades Hospitalares, nas quais as vagas disponíveis estão em constante mudança por existirem situações de urgência, internamentos e a chegada de veículos de emergência médica.

Mais uma vez, é enviada uma mensagem para o agente UGVE com as informações de disponibilidade atualizadas necessárias para permitir que a informação permaneça sempre atualizada em todo o sistema MultiAgente.

3.3.4 *usRecebeDoente*

O *behaviour usRecebeDoente* trata-se de um *behaviour* cíclico. Desta forma, estará à escuta de mensagens de confirmação do fim do transporte do doente e consequentemente chegada à US. Assim que recebe a *performative confirm_end*, a US irá verificar se o veículo que transportou o doente foi um helicóptero. Em caso afirmativo, assume-se que a partir desse momento, o heliporto está livre, efetuando-se, portanto, a atualização da disponibilidade na variável *current_state* para *True*. Desta forma, mantém-se a coerência entre a informação contida na US e na UGVE. Em caso de se tratar de outro tipo de veículo, é apenas mostrada a informação de fim de transporte do doente.

3.4 Agente Veículo

O agente veículo permite representar os diferentes veículos de emergência médica. Este tipo de agente pode representar vários tipos de veículos, nomeadamente, ambulância do INEM,

ambulância dos Bombeiros, ambulância da Cruz Vermelha e helicóptero, que serão utilizados de acordo com a gravidade da situação de emergência médica.

Este agente tem associado a si dois *behaviours* *RegisterVeiculo_Behav* e *Transport_Behav* que permitem que este desempenhe as funções delineadas inicialmente.

3.4.1 RegisterVeiculo_Behav

Este *behaviour*, do tipo *OneShot Behaviour*, é realizado apenas uma vez e permite que o veículo se registre na UGVE, isto é, que as suas informações sejam enviadas e armazenadas. Para auxiliar a troca de informação é utilizada uma classe auxiliar *InformPositionVeiculo*. É então criada uma instância desta classe com as informações como o *jid* do agente, do tipo *string*, a posição, uma instância da classe auxiliar *Position*, o tipo de veículo e a sua disponibilidade, valor do tipo booleano.

Estas informações são guardadas no próprio agente através da variável *current_location*, inicialmente com valor *None*.

O envio do objeto *InformPositionVeiculo* é feito utilizando o método *encode* da biblioteca *jsonpickle* e é utilizada a *performative subscribe_Veiculo*. Esta mensagem será posteriormente recebida pela UGVE e a informação tratada de forma a manter uma lista de veículos disponíveis para situações de emergência.

3.4.2 Transport_Behav

O *behaviour Transport_Behav* é utilizado para realizar o transporte de doentes, nomeadamente, uma primeira viagem de deslocação até ao doente e uma segunda viagem de transporte do doente até à Unidade de Saúde adequada.

Este *behaviour* recebe inicialmente uma mensagem com o pedido de transporte. Este pedido de transporte pode estar relacionado com a primeira viagem ou a segunda viagem, sendo que dependendo do tipo de *performative* que recebe, vai realizar diferentes ações.

No caso da *performative* ser *request*, o pedido recebido corresponde ao primeiro transporte. O *behaviour* recebe a informação da posição do doente e atualiza disponibilidade do veículo para *False* já que está encarregue de fazer a deslocação até ao respetivo doente. O tempo definido para cada transporte foi de 3 segundos.

No final desta primeira deslocação, o veículo altera a sua posição atual para a posição do doente, mantendo a informação atualizada. Posteriormente, o *behaviour* faz a reavaliação do doente, escolhendo uma especialidade médica para a situação observada (de forma aleatória). Assim, é enviado para UGVE um objeto *InformReavaliacao*, instância da classe auxiliar *InformReavaliacao*, com informações sobre a identificação do doente, a sua posição e a especialidade escolhida. Esta informação vai ser recolhida por parte da UGVE de forma a escolher a US adequada para tratamento do doente.

No caso da *performative* ser *inform_unidade*, a informação recebida vai estar relacionada com o segundo transporte. O *behaviour* recebe então da UGVE o objeto *UnidadeSaude* que contém informações sobre a US adequada para tratamento do doente. Depois de processar a informação recebida, vai realizar o segundo transporte levando o doente até a Unidade de Saúde. Como já referido anteriormente, este transporte tem duração de 3 segundos.

Assim que o transporte é realizado, vai ser atualizada a posição atual do veículo para a posição da US respetiva, e atualizada a sua disponibilidade para *True*, já que o segundo transporte foi concluído e o veículo está pronto para ser utilizado novamente.

Por fim, de forma a manter a informação coerente em todo o Sistema MultiAgente, vai ser enviada uma mensagem de confirmação à UGVE, ao doente, e à Unidade de Saúde a informar o fim de transporte. A UGVE receberá esta mensagem de forma a atualizar a posição e disponibilidade do veículo. Quanto ao agente doente, este receberá a mensagem, processando-a de forma a terminar as suas funções.

4. Arquitetura do sistema Multiagente

Os diagramas de UML (*Unified Modeling Language*) são utilizados para modelar sistemas MultiAgentes. Nesta secção vão ser exibidos os diagramas de Classes, Sequência, Colaboração e Atividades de forma a melhorar a compreensão do design do sistema MultiAgente desenvolvido.

Estes esquemas permitem um melhor entendimento da dinâmica e interações entre os agentes, mostrando as suas comunicações, atividades e sequências de ações.

4.1 Diagrama de Classes

Na Figura 2, está representado o diagrama de classes do sistema Multiagente em questão:

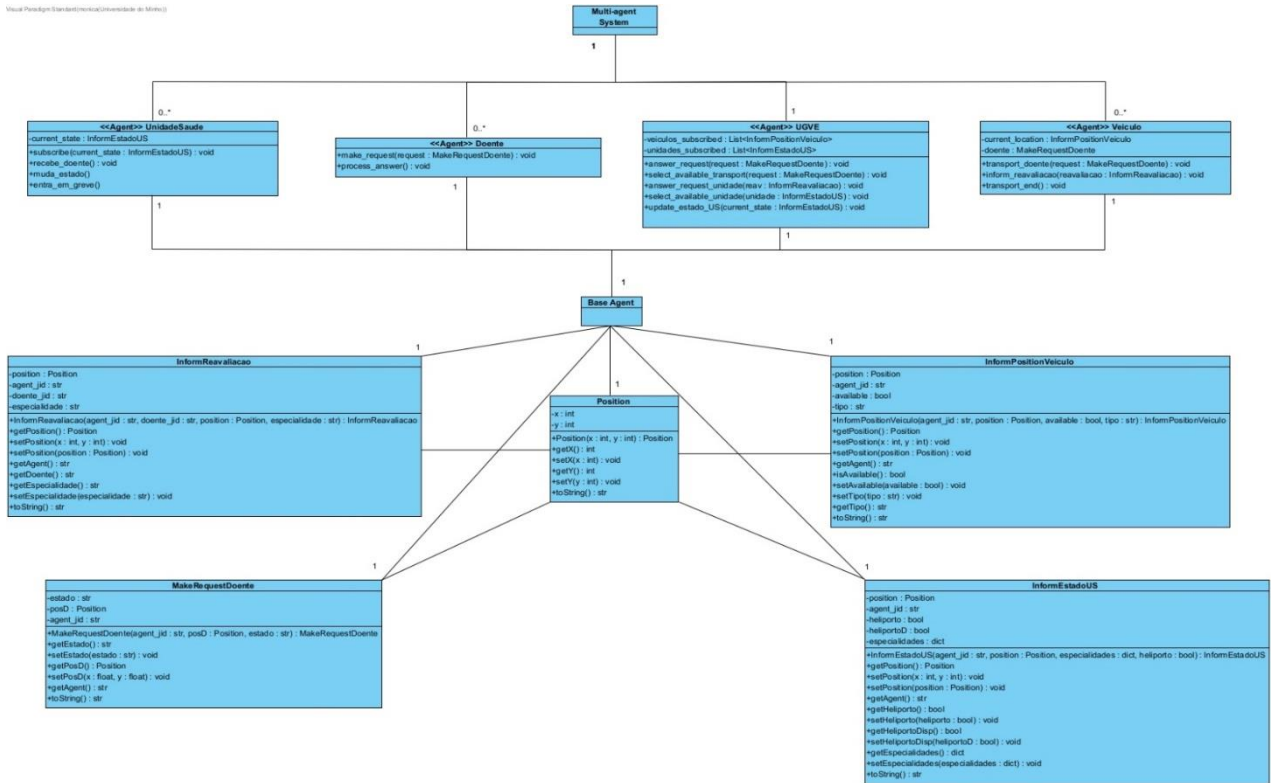


Figura 2-Diagrama de Classes do Sistema de Gestão de Veículos de Emergência.

Este diagrama permite uma representação das classes que compõem o sistema e as relações existentes entre as mesmas.

Podem verificar-se os diferentes atributos associados a cada uma das classes, necessários para a troca de informação entre os agentes, nomeadamente:

- *Position*: composto pelas coordenadas x e y para representar a localização do paciente, do veículo e da unidade de saúde;
- *InformPositionVeiculo*: os atributos desta classe dizem respeito às características do veículo de emergência, nomeadamente localização, *jid*, disponibilidade e tipo;
- *InformEstadoUS*: contém toda a informação relativa à unidade de saúde (localização, *jid*, existência de heliporto, disponibilidade do heliporto e especialidades (com respetivas vagas);
- *MakeRequestDoente*: enviado do paciente para a UGVE, com a informação do estado (que pode ser: leve, moderado, grave, muito grave ou urgente), posição e *jid* do mesmo;

- . *InformReavaliacao*: enviado do Veículo para a UGVE (quando o primeiro chegou ao paciente) com o objetivo de saber a unidade de saúde para onde se deve deslocar. É composto pelos *jid* do veículo e do paciente, posição e especialidade necessária para socorrer o doente;

Para além disso, é possível compreender o papel que cada agente tem no sistema. Observe-se, por exemplo, o agente UGVE. Este agente é responsável por inúmeras tarefas, nomeadamente o atendimento aos pedidos dos pacientes que inclui a seleção do veículo e unidade de saúde mais adequada. Quanto aos restantes agentes, as suas funções já foram explicadas anteriormente, podendo ser visualizadas no diagrama da Figura 2.

4.2 Diagrama de Colaboração

O diagrama de colaboração, apresentado na Figura 3, permite ter uma melhor perceção da interação entre os agentes envolvidos, tendo em conta as *performatives* definidas.

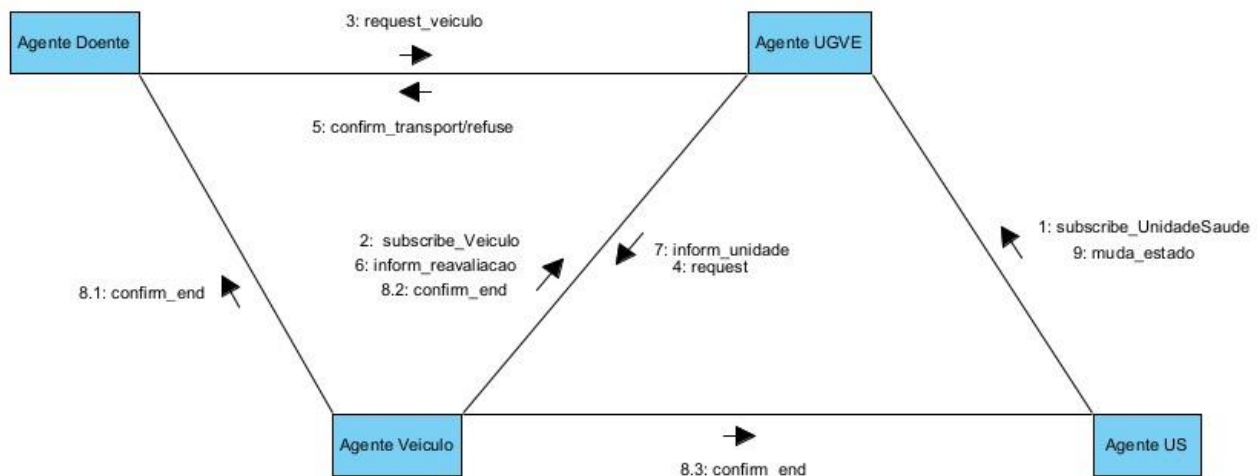


Figura 3- Diagrama de Colaboração do Sistema de Gestão de Veículos de Emergência.

Através da análise do diagrama apresentado, verifica-se que decorre o seguinte:

1. Inicialmente, são registadas as unidades de saúde na UGVE, recorrendo à *performative* *subscribe_UnidadeSaude*;
2. Os veículos são, posteriormente, registados na UGVE, utilizando a *performative* *subscribe_Veiculo*;

3. É enviado um pedido de transporte pelo doente à UGVE (*request_veiculo*);
4. A UGVE, após processar o pedido e selecionar o veículo mais adequado para o transporte, reencaminha o veículo selecionado para a localização do doente (*request*);
5. A UGVE responde, então, ao doente confirmando o transporte se disponível (*confirm_transport*) ou informando que não se encontra nenhum veículo disponível (*refuse*);
6. Ao chegar ao local, o veículo de emergência informa à UGVE a reavaliação do estado do doente utilizando a *performative inform_reavaliacao*;
7. A UGVE trata, posteriormente, de selecionar uma unidade de saúde próxima do doente que esteja preparada para o receber, comunicando essa informação ao veículo recorrendo à *performative inform_unidade*;
8. Ao concluir o transporte do doente para a unidade de saúde selecionada, o veículo envia uma confirmação ao doente (8.1), à UGVE (8.2) e à unidade de saúde (8.3) de fim de transporte, através da *performative confirm_end*;
9. Por fim, de modo a atualizar a quantidade de vagas disponíveis para determinadas especialidades de unidades de saúde, bem como para definir as mesmas como nulas em caso de greve nessa especialidade é utilizada a *performative muda_estado*, enviada das unidades de saúde para a UGVE.

4.3 Diagrama de Sequência

O diagrama de sequências descreve a forma como os agentes do sistema interagem e colaboram entre si, utilizando uma linguagem UML (*Unified Modeling Language*).

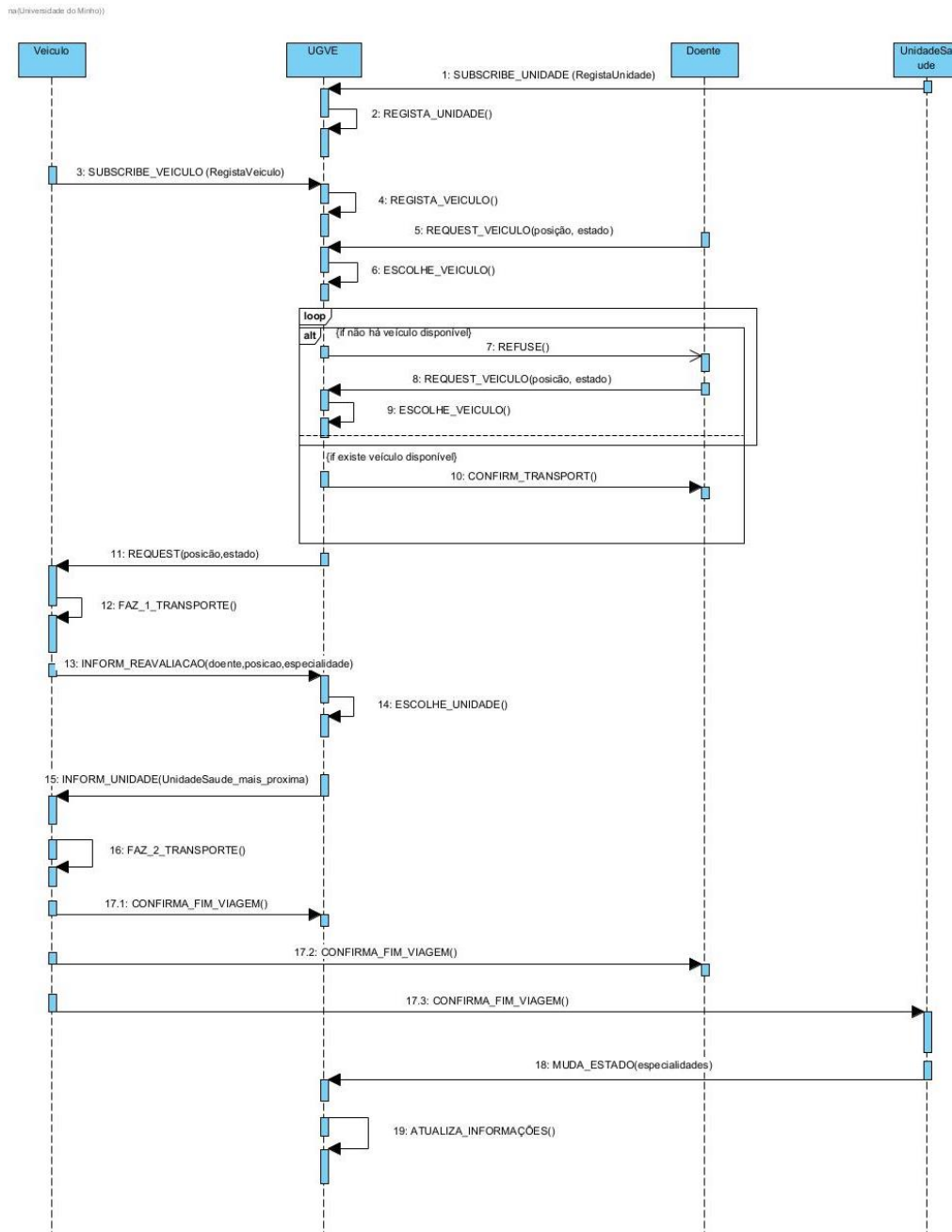


Figura 4- Diagrama de Sequências do Sistema de Gestão de Veículos de Emergência.

Analisando o diagrama da Figura 4 de uma forma mais detalhada, é possível observar as seguintes etapas:

1. O agente Unidade Saúde envia ao agente UGVE informações de geolocalização, especialidades disponíveis e respectivas vagas e existência ou não de heliporto (*SUBSCRIBE_UNIDADE(RegistaUnidade)*).
2. O agente UGVE regista a respetiva unidade hospitalar (*REGISTA_UNIDADE()*).
3. O agente Veículo envia ao agente UGVE informações de geolocalização, disponibilidade e o seu tipo (*SUBSCRIBE_VEICULO (RegistaVeiculo)*).
4. O agente UGVE regista o respetivo veículo (*REGISTA_VEICULO()*).
5. O agente Doente faz um pedido de transporte ao agente UGVE, informando a sua geolocalização e o seu estado (*REQUEST_VEICULO (posição, estado)*);
6. O agente UGVE recebe o pedido e tendo em conta o estado do doente, escolhe um veículo de um determinado tipo, estando este disponível e o mais próximo possível (*ESCOLHE_VEICULO()*);
7. Caso o agente UGVE não encontre nenhum veículo, envia uma mensagem de recusa do pedido ao agente Doente (*REFUSE()*);
8. O agente Doente envia novamente um pedido de transporte ao agente UGVE (*REQUEST_VEICULO (posição, estado)*);
9. O agente UGVE procede novamente à escolha do veículo (*ESCOLHE_VEICULO()*);
10. Caso o agente UGVE consiga selecionar um veículo de acordo com os requisitos especificados, envia uma mensagem de confirmação de transporte ao doente (*CONFIRM_TRANSPORT()*);
11. O agente UGVE envia ao veículo escolhido o pedido do doente para este executar o transporte para o local indicado (*REQUEST(posição, estado)*);
12. O agente Veículo executa o transporte, deslocando-se até ao local do doente (*FAZ_1_TRANSPORTE()*);
13. Após a chegada ao local, o agente Veículo informa o agente UGVE da reavaliação realizada, enviando o *jid* do doente, a geolocalização e a especialidade requerida (*INFORM_REAVALIACAO (doente, posição, especialidade)*);
14. O agente UGVE recebe a reavaliação do agente Veiculo e de acordo com a especialidade requerida para o doente, verifica quais unidades de saúde têm a especialidade e se têm vagas disponíveis. Posteriormente, o agente escolhe a mais próxima do doente (*CONFIRMA_TRANSPORT()*);
15. O agente UGVE envia a geolocalização da unidade de saúde mais próxima para o mesmo agente Veículo (*INFORM_UNIDADE (UnidadeSaude_mais_proxima)*);

16. O agente Veiculo executa o transporte para a unidade de saúde indicada (*FAZ_2_TRANSPORTE()*);
17. O agente Veiculo confirma o fim da viagem e chegada à unidade de saúde ao agente UGVE, UnidadeSaude e Doente (*CONFIRMA_FIM_VIAGEM()*);
18. O agente UnidadeSaude envia periodicamente atualizações do número de vagas disponíveis em cada uma das especialidades. Além das atualizações devido à saída e entrada de doentes na sua unidade, este agente envia também atualizações derivadas a greve em algumas das suas especialidades. (*MUDA_ESTADO(especialidades)*);
19. Por fim, o agente UGVE atualiza as informações do agente UnidadeSaude, garantindo a coerência e o correto funcionamento do sistema implementado (*ATUALIZA_INFORMACOES()*).

4.4 Diagrama de Atividades

O diagrama de atividades serve para modelar o aspeto comportamental dos processos do SMA. Uma atividade é uma sequência de ações, podendo passar por nós de decisão, zonas recursivas e atividades em simultâneo. Estão representados no diagrama presente na Figura 5, os 4 agentes presentes no SMA.

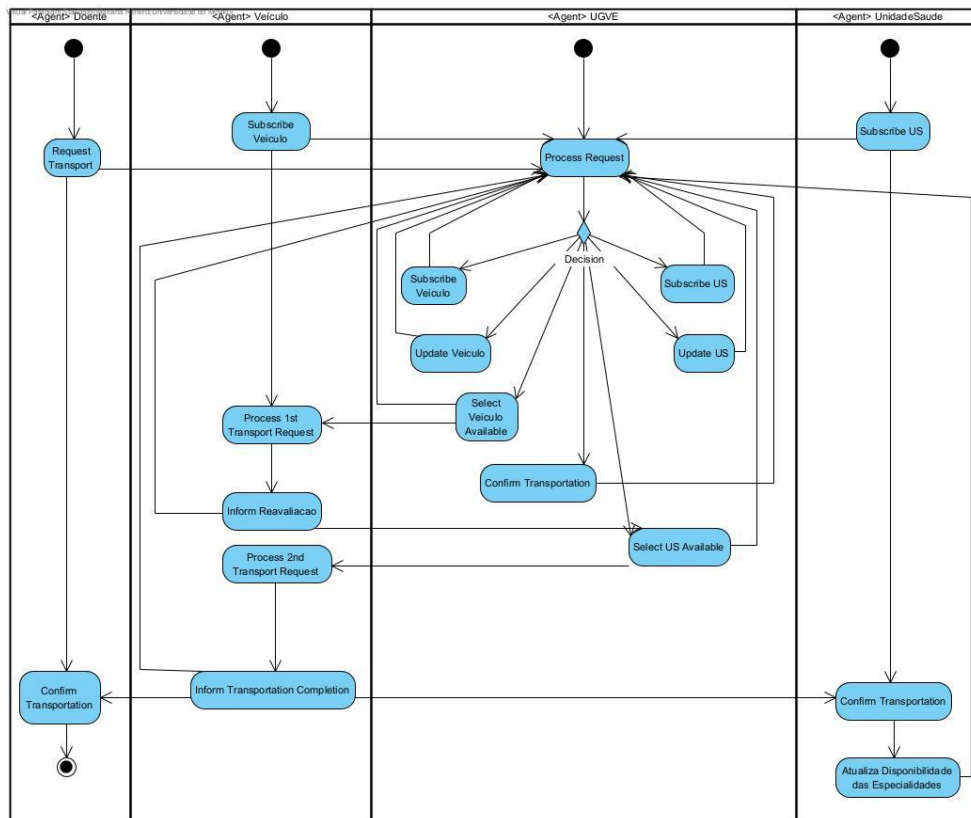


Figura 5 - Diagrama de Atividades do Sistema de Gestão de Veículos de Emergência.

O agente UGVE tem uma atividade constante de gestão de pedidos. Dependendo dos pedidos que recebe, toma decisões consoante as ações a tomar, podendo estas ser registar veículo ou unidade de saúde, atualizar as informações do veículo ou Unidade de Saúde, selecionar veículo e Unidade de Saúde disponíveis apropriadas com base em determinados critérios já referidos, e ainda confirmar o fim do transporte do doente.

Inicialmente, a Unidade de Saúde apresenta a atividade de registo na UGVE, enviando as informações necessárias e adequadas já mostradas noutras secções do relatório. Após o transporte do doente esta apresenta uma outra atividade correspondente à receção da confirmação de chegada à Unidade de Saúde. Posteriormente, ocorre também a atividade de atualização da disponibilidade das especialidades, quer em situação de greve quer na entrada e saída de pacientes.

No caso do veículo, à semelhança da Unidade de Saúde este regista-se na UGVE, enviando as suas informações iniciais. Posteriormente, este recebe o primeiro pedido de transporte, correspondente à deslocação até ao doente, levando à atividade de informar a reavaliação médica feita junto do doente. Depois da atividade de seleção de Unidade de Saúde feita pela UGVE, o veículo recebe, consequentemente, a informação sobre o segundo pedido de transporte, fazendo então a deslocação do doente até a posição da Unidade de Saúde. Finalmente, quando este segundo transporte está completado, o veículo faz a ação de confirmação de fim de viagem para o doente, a UGVE e a Unidade de Saúde.

Por fim, o doente apresenta apenas duas atividades, correspondentes a fazer o pedido de transporte à UGVE, e receção da confirmação de final do seu transporte até à Unidade de Saúde.

5. Implementação do Sistema MultiAgente

Para inicializar o SMA, foi criado um ficheiro *Python* chamado *main* para que fosse possível definir quantos agentes de cada tipo existiriam.

Deste modo, o primeiro agente a ser criado e inicializado é o agente UGVE. Uma vez que este agente irá receber as subscrições dos veículos e das unidades de saúde, é necessário que seja criado em primeiro lugar para suportar a receção dessas mensagens.

Quanto aos restantes agentes, foram criadas 3 listas para guardar todos os agentes dos diferentes tipos (veículos, doentes e unidades de saúde). Assim, uma vez terminado o agente UGVE, estas listas são percorridas e todos os agentes acabam as suas funções.

Quanto às unidades de saúde, um ciclo *for* é criado para criar tantas unidades quantas forem definidas na variável `MAX_UNIDADES`. O *jid* destes agentes tem como formato

usX@XMPP_SERVER, sendo X o número da unidade de saúde e *XMPP_SERVER* o servidor do programa associado ao protocolo XMPP que por sua vez é usado em programas de mensagens instantâneas e comunicação em tempo real. Criado o agente, é adicionado o seu *jid* à lista *us_agents_list*.

Os agentes correspondentes aos doentes são criados de forma semelhante e a quantidade dos mesmos é definida na variável *MAX_DOENTES*. O *jid* destes agentes tem, por exemplos, o seguinte formato: *doente1@XMPP_SERVER*.

A criação dos agentes Veículo separou-se em duas partes: uma parte para a criação de helicópteros e outra para a criação dos restantes veículos. Esta separação foi efetuada porque a escolha do tipo de veículos é feita aleatoriamente dentro de uma lista de veículos possíveis (["ambulânciaINEM", "ambulânciaBombeiros", "ambulânciaCruzVermelha"]). Assim, para garantir a existência de helicópteros no SMA, realizaram-se os ciclos *for* mencionados anteriormente de forma separada.

Por fim, é definido um ciclo *while* infinito que garante a continuação do sistema até que haja uma interrupção do mesmo.

6. Conclusão e Melhorias Futuras

Em suma, o SMA desenvolvido permite a gestão do transporte de doentes através de veículos de emergência diversos para unidades de saúde que diferem entre si, quer em especialidades disponibilizadas, bem como na quantidade de vagas existente para cada uma. Tendo sempre em conta as condições do doente e a disponibilidade de serviços, quer de transporte como de prestação de cuidados de saúde, o sistema é capaz de se adaptar apresentando uma abordagem eficiente e coordenada, permite uma resposta rápida a solicitações de transporte, considerando sempre diversas variáveis reais.

É importante referir, porém, que existem melhorias que podem ser implementadas no futuro, de modo a aproximar este sistema a uma utilização no mundo real, tornando-o mais dinâmico, como aumentar o número de doentes por veículo de emergência e a possibilidade de existência de mais do que um helicóptero para cada unidade de saúde com heliporto. Poderia ser ainda acrescentada a verificação da não necessidade de transporte do doente para uma US, sendo possível o tratamento ser feito no próprio veículo.

7. Referências Bibliográficas

[1] Novais, P., & Analide, C. *Agentes Inteligentes*. (2006).

[2] Palanca, Javi. *SPADE Documentation*. (2018).